

```
"""
```

```
Sistema de Gerenciamento de Estoque (CLI)
```

```
Este script gerencia o cadastro, exibição e cálculo de produtos em  
estoque
```

```
utilizando uma interface baseada em terminal (CLI).
```

```
"""
```

```
from tabulate import tabulate
```

```
import regex
```

```
import time
```

```
# Formatação de Texto / Estilos ANSI do Terminal
```

```
ITALIC = "\033[3m"
```

```
BOLD = "\033[1m"
```

```
RESET = "\033[0m"
```

```
# Lista principal para armazenar os dicionários dos produtos
```

```
stock = []
```

```
# Lista com mensagens de erro
```

```
error_message = {
```

```
    "menu_error": "⚠️ Por favor, escolha uma das opções válidas do  
menu.\n",
```

```
    "calculate_total_stock_error_message": "Não há produtos  
cadastrados.\n",
```

```
    "code_error_product": "⚠️ Informe um código válido para o produto  
(deverá ter até 13 números).\n",
```

```
    "name_error_product": "⚠️ Informe corretamente o nome do produto  
(deverá ter pelo menos 3 letras).\n",
```

```
    "price_error_product": "⚠️ Informe o valor correto do  
produto.\n",
```

```
    "quantity_error_product": "⚠️ Informe a quantidade correta do  
produto.\n",
```

```
}
```

```
def incorrect_option(code_error: str) -> None:
```

```
    """Exibe no terminal uma mensagem padronizada para entradas e  
opções inválidas.
```

```
    Args:
```

```
        code_error (str): Mensagem de erro específica a ser apresentada
```

ao usuário.

"""

```
print("\n!  OPÇÃO INVÁLIDA ou DIGITAÇÃO INCORRETA.")
print(code_error)
```

def display\_menu() -> int :

"""Exibe o menu principal de navegação e captura a escolha do usuário.

Returns:

int: 0 número da opção selecionada pelo usuário ou -1 em caso de entrada inválida.

"""

```
print('===== MENU =====')
print('1 - Cadastrar Produto')
print('2 - Visualizar Estoque (Tabela)' ) # <--- Nova opção!
print('3 - Calcular Total de Produtos em Estoque')
print('0 - SAIR')
print('=====')
try:
    menu_option = int(input('Informe a opção desejada: '))
    return menu_option
except ValueError:
    return -1
```

def register\_product() -> None:

"""Gerencia o fluxo completo de cadastro de um novo produto no estoque.

Solicita e valida código, nome, preço e quantidade antes de armazenar

o item no dicionário global de estoque.

"""

```
print("\n--- Cadastrar Produto ---")
code = get_product_code()
name = get_product_name()
price = get_product_price()
quantity = get_product_quantity()
```

```
new_product = {
    "code": code,
    "name": name,
    "price": price,
    "quantity": quantity
```

```

    }

    stock.append(new_product)
    print(f'ITEM: \n | Código: {code} || Nome: {name} || Preço: {price}
|| Quantidade: {quantity} |\n')
    print('...cadastrando produto...')
    time.sleep(2)
    print(f'✅ {ITALIC}PRODUTO CADASTRADO COM SUCESSO!!!{RESET}\n')

```

```

def display_stock() -> None:
    """Formata e exibe os produtos cadastrados em formato de tabela no
terminal.

```

Utiliza a biblioteca 'tabulate' para gerar a visualização do estoque.

Caso o estoque esteja vazio, exibe uma mensagem informativa de erro.

```

    """
    print("\n--- Produtos em Estoque ---")

    if not stock:
        print(error_message["calculate_total_stock_error_message"])
        return

    formatted_stock = []
    for item in stock:
        formatted_stock.append({
            "Código": item["code"],
            "Nome": item["name"],
            "Preço (R$)": f"{item['price']}" if
isinstance(item['price'], str) else f"R$
{item['price']:.2f}".replace('.', ','),
            "Quantidade": item["quantity"]
        })

    print(tabulate(formatted_stock, headers="keys",
tablefmt="fancy_grid"))
    print("=====\n")

```

```

def calculate_total_stock() -> None:
    """Calcula e exibe a quantidade total somada de itens presentes no
estoque via função lambda.

```

Soma as quantidades de todos os produtos cadastrados e exibe o resultado.

```

"""
print("\n--- Total de Produtos em Estoque ---")

if len(stock) > 0:
    stock_total = sum(map(lambda x: x["quantity"], stock))
    print(f'Valor total: {stock_total} itens. \n')
    return
else:
    print(error_message["calculate_total_stock_error_message"])

def get_product_code() -> int:
    """Solicita e valida o código de barras do produto.

    Garante que a entrada contenha apenas dígitos (até 13 caracteres),
    preenche com zeros à direita até atingir 13 dígitos (`ljust`) e
    verifica a ausência de duplicatas na lista de estoque.

    Returns:
        int: Código de barras validado e convertido para número inteiro.
    """

    while True:
        try:
            entry = input('Informe o código de barras do produto:
').strip()
            print('-----')

            if not entry.isdigit() or len(entry) > 13:
                incorrect_option(error_message["code_error_product"])
                continue

            formatted_entry = entry.ljust(13, '0')

            new_code = int(formatted_entry)

            if not new_code in list(map(lambda x: x['code'], stock)):
                return new_code
            else:
                print(f'\nO código {formatted_entry} é de um produto já
registrado. Adicione um novo código!\n')

        except ValueError:
            incorrect_option(error_message["code_error_product"])

```

```

def get_product_name() -> str:
    """Solicita e valida o nome do produto através de expressões
    regulares (Regex).

    Exige no mínimo 3 caracteres no total e pelo menos 3 letras
    (suportando Unicode/acentuação).

    Returns:
        str: Nome do produto validado.
    """

    valid_name_regex = r"^(?=(?:.*\p{L}){3,}).{3,}$"
    while True:
        try:
            new_name = input('Informe o nome do produto:
').strip()

            print('-----')

            if regex.search(valid_name_regex, new_name):
                return new_name
            else:
                incorrect_option(error_message["name_error_product"])
        except ValueError:
            incorrect_option(error_message["name_error_product"])

def get_product_price() -> str:
    """Solicita e valida o preço unitário do produto.

    Garante que o valor digitado não seja negativo e o converte para o
    formato
    monetário brasileiro (R$ X,XX).

    Returns:
        str: Preço formatado em string como moeda brasileira.
    """

    while True:
        try:
            entry = input('Informe o preço do produto: R$
').strip().replace(',', '.')
            print('-----')

            new_price = float(entry)

```

```

        if new_price < 0:

incorrect_option(error_message["price_error_product"])
                continue

                return f"R$ {new_price:.2f}".replace('.', ',')
    except ValueError:
        incorrect_option(error_message["price_error_product"])

def get_product_quantity() -> int:
    """Solicita e valida a quantidade de itens do produto.

    Garante que a entrada seja um número inteiro positivo.

    Returns:
        int: Quantidade de itens em estoque.
    """

    while True:
        try:
            entry = input('Informe a quantidade de itens do
produto: ').strip()
            print('-----')

            new_quantity = int(entry)

            if new_quantity <= 0:

incorrect_option(error_message["quantity_error_product"])
                    continue

                    return new_quantity
        except ValueError:

incorrect_option(error_message["quantity_error_product"])

# Loop principal de execução do menu
while True:
    menu_option = display_menu()
    if menu_option == 1:
        register_product()

    elif menu_option == 2:
        display_stock()

```

```
elif menu_option == 3:
    calculate_total_stock()

elif menu_option == 0:
    break

else:
    incorrect_option(error_message["menu_error"])

print('\n...ENCERRANDO O SISTEMA:')
time.sleep(2)
print(f'{ITALIC}{BOLD}Obrigado(a) por usar o nosso programa! ✨{RESET}')
```